

05 - Middleware de Autenticación JWT

Ejercicio 05 — Middleware de Autenticación con JWT



Foco: seguridad, autenticación, middlewares de Express **Tiempo estimado:** 2 clases [← Volver al README](#)

Objetivo

Que uses la IA para **agregar autenticación con JWT** sin entregar un agujero de seguridad. Este ejercicio mezcla "generar código nuevo" (fácil) con "entender qué te dio la IA en materia de seguridad" (difícil de verdad).

El problema

Hoy **cualquiera** puede hacer `DELETE /api/alumnos/3`. No hay login, no hay tokens, no hay nada. Todos los endpoints son públicos.

Queremos:

- Un endpoint `POST /api/auth/login` que reciba credenciales y devuelva un **JWT**.
 - Un **middleware** que valide el token en el header `Authorization: Bearer <token>` y rechace con **401** si falta o es inválido.
 - Proteger las operaciones de escritura (`POST`, `PUT`, `DELETE`); las de lectura (`GET`) podés dejarlas públicas o no (decisión tuya, justificala).
-

Qué tenés que lograr

1. Instalar `jsonwebtoken` (esta dependencia **sí** es esperable, pero verificá que la IA use la versión correcta).
 2. Endpoint de login que firme un token con un **secreto leído de .env** (no hardcodeado).
 3. Middleware `authMiddleware` en `src/middlewares/` que verifique el token.
 4. Aplicarlo a los endpoints que correspondan.
-

Cómo encarar el prompting

💡 Acá la IA es tu amiga porque JWT es un patrón estándar y lo conoce bien. Pero **la seguridad se rompe en los detalles**, y ahí tenés que estar fino.

Restricciones que Sí o Sí van en tu prompt:

- "El secreto del JWT tiene que leerse de `process.env.JWT_SECRET`, nunca **hardcodeado** en el código."
- "El token tiene que tener expiración (`expiresIn`)."
- "El middleware debe devolver 401 si el header falta, está mal formado, o el token es inválido/expirado — *distinguí esos casos.*"
- "Seguía la arquitectura del proyecto: middlewares en `src/middlewares/`, ES modules."

⚠️ **Trampa #1 — el secreto hardcodeado:** la IA casi siempre te pone `const SECRET = "mi-secreto-super-seguro"` en el ejemplo. **Eso es un 🔴 en seguridad.** Si lo dejás así, perdés el punto. Tiene que salir del `.env`.

⚠️ **Trampa #2 — login de mentira:** para un TP educativo está OK un login con usuario/clave fijos, **pero** la clave no puede estar en texto plano comparada con `==`. Como mínimo, dejá claro en la reflexión que en producción iría con hash (bcrypt) contra una tabla de usuarios.

⚠️ **Trampa #3 — verify vs decode:** `jwt.decode()` **NO valida la firma**, solo lee el contenido. Si la IA usa `decode` en el middleware en vez de `verify`, cualquiera puede falsificar un token. Verificalo.

🤖 **Pregunta capciosa que te puede caer:** *un JWT, ¿está encriptado?* (Respuesta: **no**. Está firmado y codificado en base64, pero el payload se lee con cualquier herramienta. Por eso no metés datos sensibles adentro.)

🔍 Verificación del resultado

- ☐ `POST /api/auth/login` con credenciales correctas devuelve un token; con incorrectas, **401**.
- ☐ `DELETE /api/alumnos/3` sin header `Authorization` devuelve **401** (no 200, no 500).
- ☐ Con un token **inválido o vencido** devuelve **401**.
- ☐ Con un token **válido** funciona normal.
- ☐ El secreto está en `.env` (revisá que no quedó hardcodeado en ningún `.js`).
- ☐ El middleware usa `jwt.verify` (no `jwt.decode`).
- ☐ El `.env` con el secreto **no está commiteado** (está en `.gitignore`).

🤖 Pregunta para el oral: *mostrame en tu código dónde se valida la firma del token. Si yo cambio una letra del token, ¿qué pasa y por qué?*

✅ Entrega

Bitácora + commit. **Obligatorio en la reflexión:** ¿qué cosas de seguridad te dio mal o "de ejemplo" la IA que tuviste que corregir? (Si decís "ninguna", probablemente no las viste.)